

Intro to AI Programming Lab Test

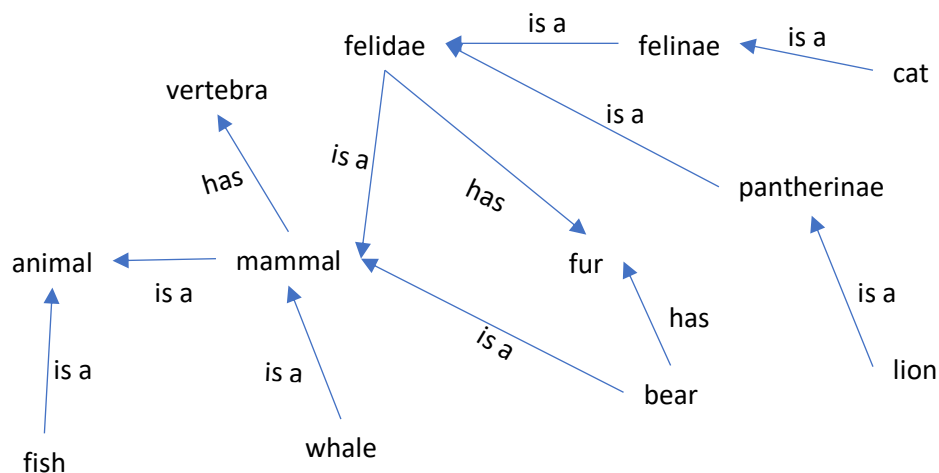
Download a Prolog text file and a Jupyter notebook. When finished, submit two files to Brightspace when finished, a single Prolog text file and the other a Jupyter notebook.

Open book test but no internet browsing or use of ChatGPT or other such tools. You can use your previous coding work but not anyone else's.

Part A - Prolog

Submit the Prolog code in a file called [labtest.txt](#). Brightspace may object to a .pl file.

0. Modify the fact **student(my_name, my_student_ID)** in your Prolog code so that it has **your name and student number** as constants.
1. Using predicates **is_a(A,B)** and **has(A,B)**, complete the representation of the following semantic network knowledgebase as Prolog facts in the Prolog code file provided:



Write a recursive predicate **isa(A,B)** which allows one to infer for example that a cat is an animal. Write a simple non-recursive rule **has_fur(X)** which is true if X has fur. For example, **has_fur(lion)** should be true. Identify all the species with fur and copy the output from Prolog interpreter to your code as a comment.

(15 marks)

2. Write a Prolog predicate **replaceOdds(L1, L2)** which replaces all odd numbers from L1 with the symbol **odd**. The mod operator may be useful.

e.g. **replaceOdds([1, 5, 4, 1, 2, 6], L2)** gives **L2 = [odd, odd, 4, odd, 2, 6]**.

(5 marks)

3. Write a recursive Prolog predicate, **dupList(L1, L2)**, that duplicates everything in a list.

e.g. **dupList([1, a, dog, [4, 1], 2], L2)** gives **L2 = [1,1,a,a,dog,dog, [4,1],[4,1],2,2]**.

(5 marks)

4. Given the various nonsensical facts about 1-way travel using bicycles, scooters and donkeys, write a recursive predicate **fromTo(A, B)** which will tell you if it's possible to get from A to B with intermediate stops if necessary and using any combination of bicycle, scooter or donkey.

How many ways can you get from Limerick to Carlow? Put your answer in as comments in your code after running it.

(5 marks)

5. If it is possible to go from A to B, you would like to know the route taken and the mode of transport at each stage as well as the total distance. To do this add a new rule **fromTo(A, B, R, D)** which is an adaptation of code from Q4 so that **R** records the route as a list and **D** the total distance.

For example, `fromTo(limerick, carlow, R, D)` gives something like:

`R = [bike(limerick, cork), bike(cork, mallow), bike(mallow, carlow)]`

`D =`

(20 marks)

Part B – Python

The Python coding and testing for this labtest is to be done in the Jupyter notebook provided.

0. Put your name and student ID as indicated on the Notebook.
1. Complete the notebook code for a simple Perceptron decision function **rescueCat(input)** to decide on whether to rescue a cat or not. Each input vector has 3 components `[x1, x2, x3]` where: `x1 = 1` means you have a dog, `0` means you don't. `x2 = 1` means you are allergic to cats, `0` means you are not. `x3 = 1` means the cat you are considering is a kitten, `0` means it is not.

You are willing to rescue the cat if any or both of the following two criteria hold, otherwise no:

- you have a dog and no cat allergy
- the cat is a kitten

Try the function on these three input scenarios:

- you have a dog, are allergic and cat is not a kitten
- you are not allergic, the cat is old and you have a dog
- the cat is a kitten, but you are allergic and do not have a dog.

(5 marks)

2. Write down as a NumPy matrix the 8 input training vectors for the problem described in Q1 with each row being an input vector. Write down the corresponding desired outputs too.

(5 marks)

3. Instead of handcrafting weights as in Q1, you are to train a SigmoidPerceptron to find good enough weights. First either copy the given Perceptron code to new cell and modify it to get a **SigmoidPerceptron** class or reuse **SigmoidPerceptron** code from earlier lab exercises. Then create a SigmoidPerceptron object whose name is your student number.

(10 marks)

4. Feedforward all the input vectors from Q2 in one go through the untrained SigmoidPerceptron and print the network's outputs.

Then train it as you did in previous lab exercises with the given training constant ***eta***. Pick the number of epochs yourself. The inputs and desired outputs are from Q2. Plot training progress if you can. Find an appropriate number of epochs for the given learning constant, ***eta***. What are the final weights? Display them.

(10 marks)

5. In the notebook there is the class **FakeNN** which initialises the weights and biases for a simple three-layer network. You want to feedforward a batch of input vectors in one go thru the network by applying the weights to outputs from previous layer and passing weighted sum thru sigmoid activation function and passing these outputs to next layer and so on.

Complete the weight & bias initialisations in the constructor and the matrix coding in `feedforward(self, xs)` to do this and then test it on the matrix of input vectors **inputs**.

Finally, using random numbers, create a corresponding matrix of target output vectors for the **inputs** in the notebook, and then calculate the mean squared cost averaged over all the input vectors.

(20 marks)